

DATABASE MANAGEMENT SYSTEMS

Complete Core Academic & Competitive Examination Study Guide

Subject: Computer Science & Engineering

Coverage: Architecture, ER Modeling, Relational Algebra, SQL, Normalization, Transactions, Concurrency Control, & Indexing

Target Level: Undergraduate Engineering & GATE Specialization

Table of Contents & Topic Roadmap

Module 1: Fundamental Concepts & Architecture	Page 2
Module 2: Entity-Relationship (ER) Data Modeling	Page 3
Module 3: Relational Data Model & Relational Algebra	Page 4
Module 4: Structured Query Language (SQL) - DDL & DML	Page 5
Module 5: Advanced SQL - Joins, Subqueries & Aggregates	Page 6
Module 6: Database Integrity Constraints & Triggers	Page 7
Module 7: Functional Dependencies & Keys	Page 8
Module 8: Relational Normalization (1NF to 3NF)	Page 9
Module 9: Advanced Normalization (BCNF, 4NF, 5NF)	Page 10
Module 10: Transaction Processing & ACID Properties	Page 11

Module 11: Concurrency Control & Serializability

Page 12

Module 12: Locking Protocols & Deadlock Handling

Page 13

Module 13: Database Recovery Techniques

Page 14

Module 14: File Structures, Indexing & B/B+ Trees

Page 15

Module 1: Fundamental Concepts & Architecture

1.1 Introduction to Database Management Systems

A **Database Management System (DBMS)** is a software platform designed to define, manipulate, retrieve, and manage data stored within a database. Prior to DBMS, organizations relied on file-processing systems. File systems presented major drawbacks including data redundancy, data inconsistency, difficulty in accessing data, data isolation, integrity problems, atomicity issues, concurrent-access anomalies, and security vulnerabilities.

1.2 Database Architecture (ANSI/SPARC Three-Schema Architecture)

The ANSI/SPARC three-schema architecture provides a structural framework to separate physical database storage from user applications. It is divided into three distinct abstraction levels:

- **External Level (View Level):** The highest level of abstraction, defining how end-users view the data. Multiple external schemas can exist tailored to specific user groups.
- **Conceptual Level (Logical Level):** Describes *what* data is stored in the database and the relationships among those data points. It hides physical storage details and focuses on entities, attributes, constraints, and relationships.
- **Internal Level (Physical Level):** The lowest level of abstraction, detailing *how* the data is physically stored on physical storage media (e.g., file organization, indexing techniques, hashing).

1.3 Data Independence

Data independence refers to the ability to modify a schema definition at one level without forcing modifications to the schema at the next higher level.

Logical Data Independence: The capacity to modify the conceptual schema without altering external schemas or application programs. Examples include adding or deleting attributes/entities.

Physical Data Independence: The capacity to modify the internal schema without altering the conceptual schema. Examples include changing record storage structures, re-indexing, or switching physical storage devices.

1.4 DBMS System Structure Components

A typical DBMS system is partitioned into several functional components:

Component Group	Sub-Component	Primary Functionality
Query Processor	DDL Interpreter	Interprets DDL statements and records definitions into the data dictionary.
	DML Compiler & Optimizer	Translates DML queries into low-level execution plans and selects optimal paths.
Storage Manager	Authorization & Integrity Manager	Enforces user privileges and checks database integrity constraints.
	Transaction Manager	Ensures the database remains consistent despite system crashes and concurrent executions.

Component Group	Sub-Component	Primary Functionality
	Buffer Manager	Fetches data from disk into main memory and decides what data to cache.

Module 2: Entity-Relationship (ER) Data Modeling

2.1 High-Level Conceptual Data Modeling

The Entity-Relationship model is a visual design framework used to represent logical real-world data structures before implementing them in a relational schema.

2.2 Fundamental ER Components

- **Entity:** A real-world object or concept distinguishable from other objects (e.g., Student, Course).
- **Entity Set:** A collection of similar entities sharing the same properties/attributes.
- **Attribute:** Properties or characteristics that describe an entity.
 - *Simple / Atomic:* Cannot be divided further (e.g., Age).
 - *Composite:* Can be divided into sub-components (e.g., Name → First_Name, Last_Name).
 - *Single-Valued vs Multi-Valued:* Multi-valued attributes hold multiple values for a single entity (e.g., Phone_Number, denoted by double ovals).
 - *Derived:* Computed from another attribute (e.g., Age derived from Date_of_Birth, denoted by dashed ovals).
- **Relationship:** An association among several entities. Degree indicates the number of participating entity sets (Binary, Ternary, n-ary).

2.3 Structural Constraints

Mapping cardinalities express the number of entities to which another entity can be associated via a relationship set:

- **One-to-One (1:1):** An entity in A is associated with at most one entity in B.
- **One-to-Many (1:N):** An entity in A is associated with any number of entities in B; an entity in B is associated with at most one entity in A.
- **Many-to-One (N:1):** Multiple entities in A associated with at most one entity in B.
- **Many-to-Many (N:M):** Entities in A and B can be associated with any number of entities in each other.

Participation Constraints:

- **Total Participation (Existence Dependency):** Every entity in the entity set must participate in at least one relationship instance (represented by double lines).
- **Partial Participation:** Some entities in the entity set may not participate in any relationship instance.

2.4 Weak Entity Sets

An entity set that does not possess a primary key of its own is termed a **Weak Entity Set**. It depends on an *Identifying Entity Set* (Strong Entity Set) via an *Identifying Relationship* (double diamond). A weak entity set is uniquely identified using the primary key of its strong entity set combined with its own **Partial Key (Discriminator)** (underlined with a dashed line).

2.5 Extended ER (EER) Features

EER introduces advanced modeling constructs:

- **Specialization:** A top-down design process identifying lower-level sub-entities within a high-level entity set.

- **Generalization:** A bottom-up design synthesis combining multiple entity sets sharing common features into a higher-level entity set.
- **Aggregation:** Abstraction model where relationship sets are treated as higher-level entities to participate in other relationships.

Module 3: Relational Data Model & Relational Algebra

3.1 Relational Model Concepts

Proposed by E.F. Codd in 1970, the relational model represents data as two-dimensional tables called **Relations**. A row in a relation is termed a **Tuple**, while a column is an **Attribute**. The set of allowed values for an attribute is its **Domain**. The number of tuples is the *Cardinality*, and the number of attributes is the *Degree (Arity)*.

3.2 Relational Algebra Fundamentals

Relational Algebra is a procedural query language that operates on one or more relations to produce a new relation. Operators are categorized into basic set-theoretic and procedural relational operators.

3.2.1 Core Operators

- **Selection (σ):** Filters tuples satisfying a specified predicate condition.

$$\sigma_{\text{predicate}}(R)$$

- **Projection (π):** Selects specified attributes and eliminates duplicate tuples across selected columns.

$$\pi_{A_1, A_2, \dots, A_n}(R)$$

- **Union ($\cup$):** Combines tuples from two union-compatible relations **R** and **S** (same arity and compatible domains).

$$R \cup S$$

- **Set Difference ($-$):** Returns tuples present in **R** but absent in **S**.

$$R - S$$

- **Cartesian Product ($\times$):** Combines every tuple of **R** with every tuple of **S**. Arity = arity(R) + arity(S).

$$R \times S$$

- **Rename (ρ):** Renames relation **R** or its attributes.

$$\rho_{X(A_1, A_2, \dots)}(R)$$

3.2.2 Derived Operators

- **Set Intersection ($\cap$):** Returns tuples common to both **R** and **S**.

$$R \cap S = R - (R - S)$$

- **Theta Join ($\bowtie_{\theta}$):** Combines Cartesian product and Selection based on predicate θ .

$$R \bowtie_{\theta} S = \sigma_{\theta}(R \times S)$$

- **Natural Join ($\bowtie$):** Performs a join over common attribute names, removing duplicate columns.
- **Division ($\div$):** Used for queries involving "for all" constraints (e.g., students enrolled in all courses).

Module 4: Structured Query Language (SQL) - DDL & DML

4.1 SQL Overview & Classification

SQL is the standard declarative language for interacting with Relational Database Management Systems (RDBMS). Commands are structured into major sub-languages:

- **DDL (Data Definition Language):** Defines schema structures (CREATE, ALTER, DROP, TRUNCATE).
- **DML (Data Manipulation Language):** Manages data instances (INSERT, UPDATE, DELETE, SELECT).
- **DCL (Data Control Language):** Manages permissions (GRANT, REVOKE).
- **TCL (Transaction Control Language):** Manages transaction state (COMMIT, ROLLBACK, SAVEPOINT).

4.2 Data Definition Language (DDL) Syntax & Schema Creation

Creating tables requires defining explicit data types and integrity constraints.

```
-- Table Creation: Departments
CREATE TABLE Departments (
    DeptID INT PRIMARY KEY,
    DeptName VARCHAR(50) NOT NULL UNIQUE,
    Location VARCHAR(100) DEFAULT 'Main Campus'
);

-- Table Creation: Employees with Foreign Keys
CREATE TABLE Employees (
    EmpID INT PRIMARY KEY,
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(50) NOT NULL,
    Salary DECIMAL(10, 2) CHECK (Salary > 0),
    DeptID INT,
    HireDate DATE DEFAULT CURRENT_DATE,
    CONSTRAINT fk_emp_dept FOREIGN KEY (DeptID)
        REFERENCES Departments(DeptID)
        ON DELETE SET NULL
        ON UPDATE CASCADE
);
```

4.3 Data Manipulation Language (DML) Core Operations

```
-- Inserting Data into Tables
INSERT INTO Departments (DeptID, DeptName, Location)
VALUES (101, 'Computer Science', 'Building A');

INSERT INTO Employees (EmpID, FirstName, LastName, Salary, DeptID)
VALUES (1, 'Ankush', 'Singh', 85000.00, 101);

-- Updating Records with Conditions
UPDATE Employees
SET Salary = Salary * 1.10
WHERE DeptID = 101;
```

```
-- Deleting Records safely  
DELETE FROM Employees  
WHERE EmpID = 1;
```

Module 5: Advanced SQL - Joins, Subqueries & Aggregates

5.1 Aggregation & Grouping Constructs

SQL aggregate functions (SUM, AVG, COUNT, MIN, MAX) operate on sets of values to return single scalar summaries. The GROUP BY clause partitions data, while HAVING filters grouped records.

```
-- Query: Find departments with average salary exceeding $70,000
SELECT DeptID, COUNT(EmpID) AS Total_Employees, AVG(Salary) AS Avg_Salary
FROM Employees
GROUP BY DeptID
HAVING AVG(Salary) > 70000;
```

5.2 Relational Joins in SQL

Joins allow data retrieval from multiple tables simultaneously based on relational keys.

Join Type	SQL Syntax Keyword	Behavior & Resulting Tuple Set
Inner Join	INNER JOIN	Returns only matching tuples from both relations based on join predicate.
Left Outer Join	LEFT JOIN	Returns all tuples from left table, plus matching tuples from right table (NULL filled if no match).
Right Outer Join	RIGHT JOIN	Returns all tuples from right table, plus matching tuples from left table.
Full Outer Join	FULL JOIN	Returns all tuples when there is a match in either left or right table.

```
-- Comprehensive Join Query Example
SELECT E.EmpID, E.FirstName, D.DeptName
FROM Employees E
LEFT OUTER JOIN Departments D ON E.DeptID = D.DeptID;
```

5.3 Subqueries & Nested Queries

Subqueries are queries nested inside a main SELECT, INSERT, UPDATE, or DELETE statement.

```
-- Non-Correlated Subquery: Find employees earning above average
SELECT FirstName, Salary
FROM Employees
WHERE Salary > (SELECT AVG(Salary) FROM Employees);

-- Correlated Subquery: Find employees earning most in their respective department
SELECT E1.FirstName, E1.DeptID, E1.Salary
FROM Employees E1
WHERE E1.Salary = (
    SELECT MAX(E2.Salary)
```

```
FROM Employees E2  
WHERE E2.DeptID = E1.DeptID  
);
```

Module 6: Database Integrity Constraints & Triggers

6.1 Domain & Key Constraints

Integrity constraints ensure database consistency and prevent invalid data entry.

- **Domain Constraint:** Restricts attribute values to valid data types, ranges, or enumerated sets (e.g., `CHECK (Age >= 18)`).
- **Entity Integrity Constraint:** Mandates that no primary key attribute value can be NULL, ensuring unique identification for every tuple.
- **Referential Integrity Constraint:** Ensures that a foreign key value must either match a valid primary key value in the referenced relation or be NULL.

6.2 Foreign Key Referential Actions

When referenced tuples in a parent table are updated or deleted, referential actions dictate child table behavior:

- **CASCADE:** Delete/Update matching tuples automatically in the child table.
- **SET NULL:** Set child foreign key attributes to NULL upon parent deletion/update.
- **SET DEFAULT:** Set child foreign key attributes to their default values.
- **RESTRICT / NO ACTION:** Reject the update or deletion in the parent table.

6.3 Active Databases: SQL Triggers

A **Trigger** is a procedural code segment automatically executed (triggered) in response to specified DML/DDI events on a table.

```
-- Example: Audit Log Trigger on Salary Updates
CREATE TABLE SalaryAudit (
    EmpID INT,
    OldSalary DECIMAL(10,2),
    NewSalary DECIMAL(10,2),
    ChangeDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TRIGGER trg_SalaryChange
AFTER UPDATE ON Employees
FOR EACH ROW
WHEN (OLD.Salary <> NEW.Salary)
BEGIN
    INSERT INTO SalaryAudit(EmpID, OldSalary, NewSalary)
    VALUES (:OLD.EmpID, :OLD.Salary, :NEW.Salary);
END;
```

Module 7: Functional Dependencies & Keys

7.1 Functional Dependency (FD) Definition

A Functional Dependency $X \rightarrow Y$ (read: X functionally determines Y) between two attribute sets X and Y in relation R specifies that whenever two tuples t_1 and t_2 agree on X , they must also agree on Y .

$$\forall t_1, t_2 \in R: \text{if } t_1[X] = t_2[X], \text{ then } t_1[Y] = t_2[Y]$$

7.2 Armstrong's Axioms

Armstrong's Axioms are a sound and complete set of inference rules used to derive all functional dependencies implied by a set F .

Axiom Category	Rule Name	Formal Mathematical Statement
Primary Axioms	Reflexivity	If $Y \subseteq X$, then $X \rightarrow Y$.
	Augmentation	If $X \rightarrow Y$, then $XZ \rightarrow YZ$ for any Z .
	Transitivity	If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$.
Secondary Rules	Union	If $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$.
	Decomposition	If $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$.
	Pseudo-transitivity	If $X \rightarrow Y$ and $WY \rightarrow Z$, then $WX \rightarrow Z$.

7.3 Attribute Closure Computation

The attribute closure of X under dependency set F (denoted X^+) is the set of all attributes functionally determined by X .

Algorithm to compute Closure X^+ :

- Initialize $X^+ = X$.
- Repeat until no more attributes can be added:
For each dependency $Y \rightarrow Z$ in F :
If $Y \subseteq X^+$, then set $X^+ = X^+ \cup Z$.
- Output X^+ .

7.4 Super Keys & Candidate Keys Determination

- **Super Key:** An attribute set K such that K^+ contains all attributes of relation R .
- **Candidate Key:** A minimal super key (no proper subset of K is a super key).
- **Prime Attribute:** An attribute that is part of any candidate key. Non-prime attributes do not belong to any candidate key.

Module 8: Relational Normalization (1NF to 3NF)

8.1 Purpose of Normalization

Normalization structures relation schemas to minimize data redundancy and prevent insertion, deletion, and update anomalies. It decomposes unsatisfactory relations into smaller, well-structured relations.

8.2 First Normal Form (1NF)

A relation R is in **1NF** if and only if all attribute values are atomic (indivisible) scalar values, and there are no repeating groups or multi-valued attributes.

8.3 Second Normal Form (2NF)

A relation R is in **2NF** if it is in 1NF and no non-prime attribute is partially dependent on any candidate key of R .

Partial Dependency: A functional dependency $X \rightarrow Y$ where X is a proper subset of a candidate key and Y is a non-prime attribute. 2NF eliminates partial functional dependencies.

8.4 Third Normal Form (3NF)

A relation R is in **3NF** if it is in 2NF and no non-prime attribute is transitively dependent on any candidate key.

Formal 3NF Rule: For every non-trivial functional dependency $X \rightarrow Y$ in F :
Either X is a **Super Key**, OR Y is a **Prime Attribute**.

8.5 Normalization Example: Step-by-Step

Consider relation $R(A, B, C, D)$ with FDs: $F = \{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$.

- Closure of A : $A^+ = \{A, B, C, D\}$. Thus, A is the candidate key.
- Prime attribute: $\{A\}$. Non-prime attributes: $\{B, C, D\}$.
- Check 2NF: All FDs have A or B on LHS. A is full key, but in $B \rightarrow C$, B is not a proper subset of candidate key A . It is in 2NF.
- Check 3NF: In $B \rightarrow C$, B is not a super key, and C is not prime. Violates 3NF!
- Decompose into 3NF: $R_1(B, C)$ with $B \rightarrow C$, and $R_2(A, B, D)$ with $A \rightarrow B, A \rightarrow D$.

Module 9: Advanced Normalization (BCNF, 4NF, 5NF)

9.1 Boyce-Codd Normal Form (BCNF)

BCNF is a stricter version of 3NF that eliminates all redundancy based on functional dependencies. A relation R is in BCNF if for every non-trivial FD $X \rightarrow Y$, X must be a super key.

Normal Form	Condition for Non-Trivial Dependency $X \rightarrow Y$	Eliminated Anomalies
1NF	Attributes must contain atomic values only.	Unstructured arrays / repeating groups
2NF	No non-prime attribute dependent on proper subset of key.	Partial Dependency Anomalies
3NF	X is Super Key OR Y is Prime Attribute.	Transitive Dependency Anomalies
BCNF	X MUST be a Super Key.	All Functional Dependency Redundancy

9.2 Lossless-Join & Dependency Preservation

Decomposing a relation R into sub-relations R_1 and R_2 must satisfy structural guarantees:

- **Lossless-Join Decomposition:** Guarantees no spurious tuples are created when joining decomposed tables.

$$\text{Condition: } (R_1 \cap R_2) \rightarrow R_1 \text{ OR } (R_1 \cap R_2) \rightarrow R_2$$

- **Dependency Preservation:** Guarantees that checking dependencies can be done locally within individual sub-relations without taking joins.

$$\text{Condition: } (F_1 \cup F_2)^+ = F^+$$

Trade-off Note: Any relational schema can be decomposed into 3NF while achieving both *Lossless-Join* and *Dependency Preservation*. However, BCNF guarantees Lossless-Join, but does NOT always guarantee Dependency Preservation.

9.3 Higher Normal Forms

- **Fourth Normal Form (4NF):** Eliminates Multi-Valued Dependencies (MVDs: $X \twoheadrightarrow Y$).
- **Fifth Normal Form (5NF / PJNF):** Eliminates Join Dependencies (JDs) not implied by candidate keys.

Module 10: Transaction Processing & ACID Properties

10.1 Transaction Concept

A **Transaction** is a logical unit of database processing that includes one or more database access operations (read, write, update, delete). It transforms the database from one consistent state to another.

10.2 ACID Properties

To ensure data integrity, database systems enforce four foundational transaction properties:

- **Atomicity:** "All or Nothing". Either all operations of the transaction execute successfully to completion, or none are applied. Managed by the *Recovery Manager*.
- **Consistency:** Preserves total database correctness. Execution of a transaction in isolation takes the database from one valid state to another. Responsibility of application programmer & constraint enforcer.
- **Isolation:** Concurrent transactions execute independently without interfering with one another. Intermediate states are invisible. Managed by the *Concurrency Control Manager*.
- **Durability:** Once a transaction commits, its structural changes persist permanently in storage, even in the event of system failure. Managed by the *Recovery Manager*.

10.3 Transaction State Transition Model

During lifecycle execution, transactions move through explicit operational states:

State	Operational Description
Active	Initial state; transaction stays here while executing read/write operations.
Partially Committed	Final statement executed, but updates reside in memory buffers (not written to disk).
Committed	Transaction finishes successfully; changes permanently written to log and disk.
Failed	Normal execution can no longer proceed due to hardware/software/logic errors.
Aborted	Transaction rolled back; database restored to state prior to transaction start.

Module 11: Concurrency Control & Serializability

11.1 Concurrency Anomalies

When multiple transactions execute concurrently without control, data anomalies occur:

- **Dirty Read (Uncommitted Dependency):** T_2 reads data written by T_1 before T_1 commits; T_1 later aborts.
- **Lost Update:** Two transactions concurrently read and write the same item, causing one update to overwrite the other.
- **Unrepeatable (Inconsistent) Read:** T_1 reads data, T_2 updates that data and commits, T_1 re-reads the data and finds a different value.

11.2 Serializability Theory

A schedule S represents the execution sequence of interleaved operations from multiple transactions. A schedule is **Serializable** if it produces the same result as some serial execution of those transactions.

11.2.1 Conflict Serializability

Two operations o_i and o_j in a schedule conflict if:

1. They belong to different transactions ($T_i \neq T_j$).
2. They access the exact same data item.
3. At least one of the operations is a write operation ($\text{Write}(X)$).

11.3 Testing for Conflict Serializability (Precedence Graph)

To test if schedule S is conflict serializable, construct a **Precedence Graph (Serialization Graph)** $G = (V, E)$:

- Nodes V : All transactions participating in schedule S .
- Directed Edge $T_i \rightarrow T_j$: Created if an operation of T_i conflicts with a subsequent operation of T_j .

Theorem: Schedule S is conflict serializable if and only if its Precedence Graph contains **NO CYCLES**.

Module 12: Locking Protocols & Deadlock Handling

12.1 Lock-Based Protocols

Locks manage concurrent access to data items. Two basic lock modes:

- **Shared Lock (S):** Required for Read operations. Multiple transactions can hold shared locks on the same item concurrently.
- **Exclusive Lock (X):** Required for Write operations. Only one transaction can hold an exclusive lock; blocks all other lock requests.

12.2 Two-Phase Locking Protocol (2PL)

2PL guarantees conflict serializability by forcing transactions to acquire and release locks in two distinct phases:

- **Growing Phase:** Transaction may acquire locks, but cannot release any lock.
- **Shrinking Phase:** Transaction may release locks, but cannot acquire new locks.

Strict 2PL: All exclusive locks held by a transaction must be retained until the transaction commits or aborts. Prevents cascading rollbacks.

Rigorous 2PL: All locks (Shared and Exclusive) held by a transaction are retained until commit/abort.

12.3 Deadlock Management

A deadlock occurs when two or more transactions are in a circular wait state, each waiting for a lock held by the other.

12.3.1 Deadlock Prevention Strategies

Uses transaction timestamps (start time) to determine priority when lock conflicts arise:

- **Wait-Die (Non-preemptive):** Older transaction waits for younger transaction; younger transaction dies (aborts) if it requests a lock held by older.
- **Wound-Wait (Preemptive):** Older transaction "wounds" (forces abort of) younger transaction; younger transaction waits if requesting lock held by older.

12.3.2 Deadlock Detection & Recovery

Maintains a **Wait-For Graph (WFG)** where directed edge $T_i \rightarrow T_j$ indicates T_i is waiting for a lock held by T_j . Periodic background threads check for cycles in WFG. When detected, recovery involves selecting a *victim transaction* to abort and roll back.

Module 13: Database Recovery Techniques

13.1 Failure Classification

- **Transaction Failure:** Logical errors or internal system exceptions (e.g., deadlock).
- **System Crash:** Hardware or power failures causing volatile main memory loss.
- **Disk Failure:** Physical storage damage causing persistent non-volatile data loss.

13.2 Log-Based Recovery Schemes

The system maintains a transaction log on stable storage recording every modification. Uses **Write-Ahead Logging (WAL)** protocol: log records must be flushed to stable disk before actual database items are updated.

13.2.1 Deferred Update Technique

Modifications are delayed until the transaction reaches its commit point. Reads occur from memory/log. Does not require UNDO capability during recovery; only REDO is executed for committed transactions.

13.2.2 Immediate Update Technique

Database modifications are written to disk while transaction is active. Requires both UNDO (for uncommitted transactions at crash time) and REDO (for committed transactions). Operates under *STEAL / NO-FORCE* buffer policies.

13.3 Checkpointing Mechanism

Checkpoints reduce log processing overhead during recovery by periodically flushing modified buffers and writing a [CHECKPOINT] record to the log.

ARIES Recovery Algorithm (3-Pass Model):

1. **Analysis Pass:** Scans log forward from last checkpoint to identify active transactions & dirty pages.
2. **REDO Pass:** Scans forward to reapply all logged changes (restoring database to state at crash).
3. **UNDO Pass:** Scans backward to reverse changes of transactions active at crash time.

Module 14: File Structures, Indexing & B/B+ Trees

14.1 Physical File Organization

Database records are mapped into fixed-size disk blocks. File structures determine performance:

- **Heap Files:** Unordered records; fast insertion ($O(1)$), slow linear search ($O(N)$).
- **Sequential Files:** Sorted by search key; fast range queries, costly insertions.
- **Hash Files:** Hash function maps search keys directly to bucket addresses; direct access ($O(1)$).

14.2 Indexing Concepts

Indexes are auxiliary data structures that accelerate search retrieval times without linear scans.

Index Category	Structural & Operational Characteristics
Primary Index	Built on an ordered key field (Primary Key) of a sequentially ordered data file. Sparse index.
Clustering Index	Built on an ordered non-key field of a sequentially ordered file. Data grouped by non-key values.
Secondary Index	Built on an unordered data file or non-ordering key field. Dense index containing pointers to every record.

14.3 Multi-Level Indexing: B-Trees & B+ Trees

Self-balancing search tree data structures optimized for block-based physical disk storage.

14.3.1 B-Tree Properties

Every node contains search keys and data record pointers. All leaf nodes reside at the exact same depth level.

14.3.2 B+ Tree Structure (Standard RDBMS Storage Engine Index)

Key structural enhancements over standard B-Trees:

- Internal non-leaf nodes contain only search key values and child pointers (routing nodes).
- **All data pointers/records reside exclusively in leaf nodes.**
- Leaf nodes are linked sequentially via a singly or doubly linked list, enabling exceptionally fast sequential scans and range queries.

B+ Tree Order Formula:

For block size **B**, pointer size **P**, key size **K**:

Maximum order **p** of internal node satisfies: $p \times P + (p - 1) \times K \leq B$